# Diseño, desarrollo y pruebas de aplicaciones web

**Breve descripción:**

Este componente formativo aborda los pilares técnicos y conceptuales para desarrollar soluciones web modernas. Integra el ciclo de vida del software, metodologías de desarrollo, técnicas de prototipado, contenidos multimedia, maquetación y administración de datos. Además, comprende la formulación de planes de pruebas, criterios de aceptación (DoR / DoD) y protocolos de seguridad de la información.

**Septiembre 2026**

## Tabla de contenido


Introducción ..... 4

1. Pruebas de software ..... 7

1.1. Concepto de pruebas de software ..... 7

1.2. Tipos de pruebas ..... 10

1.3. Plan de pruebas ..... 14

1.4. Criterios de aceptación ..... 17

1.5. Definition of Ready y Definition of Done ..... 20

2. Prototipado de aplicaciones ..... 25

2.1. Concepto de prototipo ..... 25

2.2. Sketch y mockups ..... 27

2.3. Wireframe ..... 28

2.4. Wireflow ..... 30

3. Desarrollo de software ..... 33

3.1. Concepto de ciclo de vida del software ..... 33

3.2. Metodologías de desarrollo ..... 34

3.3. Ejecución de la metodología ..... 37

3.4. Fases del desarrollo de software ..... 39

4. Contenido web ..... 42

4.1. Características y distribución del contenido web .....42
4.2. Formatos y tipos de archivo .....44
4.3. Imágenes, fotografías y vínculos .....46
4.4. Diseño visual y factores del diseño visual .....48
4.5. Licencias para el uso y publicación de recursos .....50
4.6. Configuración y administración de bases de datos de contenido web .....52
5. Diseño y desarrollo web .....55
5.1. Procedimientos y protocolos de diseño y desarrollo web .....55
5.2. Técnicas de diseño y estructura visual .....58
5.3. Protocolos de seguridad de la información .....61
5.4. Técnicas de maquetación .....63
5.5. Parámetros de visualización .....65
Síntesis .....68
Glosario .....69
Referencias bibliográficas .....71
Créditos .....72

# Introducción

En el panorama tecnológico actual, la construcción de aplicaciones web ha dejado de ser un proceso puramente artesanal de escritura de código para convertirse en una disciplina de ingeniería de software estructurada, multidisciplinaria y orientada a la calidad. La demanda de plataformas accesibles, seguras, escalables y altamente usables exige que los desarrolladores dominen no solo la sintaxis de lenguajes de programación o maquetación, sino también todo el flujo de trabajo que garantiza el éxito del producto digital.

Este componente formativo permite desarrollar las competencias necesarias para gestionar de manera sistemática cada etapa de la creación de aplicaciones web. Inicia con la comprensión del ciclo de vida del desarrollo de software y las metodologías ágiles que rigen la industria moderna. Posteriormente, se profundiza en la fase de experiencia e interfaz de usuario (UX / UI) mediante la elaboración de bocetos (Sketches), esquemas de baja fidelidad (wireframes), maquetas de alta fidelidad (mockups) y flujos navegables (wireflows).

Asimismo, se desarrollan competencias para administrar contenidos multimedia, manejar licencias de uso, configurar bases de datos para sistemas web y aplicar estándares de maquetación limpia y segura. Finalmente, se hace especial énfasis en la cultura del aseguramiento de la calidad (Quality Assurance - QA), mediante el diseño de planes de pruebas, la definición de criterios de aceptación claros y la adopción de conceptos fundamentales en marcos ágiles como la definición de listo (definition of ready) y la definición de hecho (definition of done), lo que contribuye a asegurar la entrega de software confiable y alineado con las necesidades del cliente.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

**Video 1.** Diseño, desarrollo y pruebas de aplicaciones web




Enlace de reproducción del video

| **Transcripción del video.** Diseño, desarrollo y pruebas de aplicaciones web |
| --- |
| Construir una aplicación web implica mucho más que lograr que una interfaz se vea bien y responda a una acción. Detrás de cada interacción existe un proceso en el que planificar, diseñar, desarrollar y comprobar se convierten en decisiones fundamentales para transformar una necesidad en una solución funcional y confiable.<br><br>Antes de escribir código, es necesario comprender qué se quiere construir y anticipar cómo interactuará el usuario con la solución. Las ideas pueden representarse, analizarse y perfeccionarse antes de convertirse en funcionalidades, permitiendo detectar oportunidades de mejora desde etapas tempranas. |

| **Transcripción del video.** Diseño, desarrollo y pruebas de aplicaciones web |
| --- |
| Sin embargo, desarrollar también significa tomar decisiones técnicas. La organización del contenido, el manejo de los datos, la estructura visual y la seguridad deben trabajar de manera articulada para conseguir una experiencia coherente, accesible y eficiente.<br><br>Y una aplicación no está realmente preparada solo porque funciona. Es necesario comprobar su comportamiento, establecer condiciones para determinar cuándo una funcionalidad está lista e identificar posibles fallos antes de que afecten al usuario. Así, la calidad deja de ser una revisión final y se integra al proceso de desarrollo.<br><br>En este componente, estos desafíos se abordarán mediante conceptos técnicos, representaciones visuales, ejemplos y situaciones aplicadas, relacionando el prototipado, el desarrollo y las pruebas con las decisiones que intervienen en la construcción de aplicaciones web.<br><br>El propósito será comprender cómo cada decisión influye en el resultado final y cómo la integración de estos elementos permite construir soluciones más sólidas y confiables. |

# 1. Pruebas de software

El aseguramiento de la calidad (Quality Assurance – QA) y el control de calidad (Quality Control – QC) constituyen pilares fundamentales e insustituibles en la ingeniería de software moderna. En las metodologías de desarrollo contemporáneas, las pruebas de software han dejado de concebirse como una fase tardía, aislada o un simple trámite administrativo previo al despliegue. Por el contrario, se ejecutan como un proceso continuo, sistemático y transversal que acompaña al software durante todo su ciclo de vida (Software Development Life Cycle – SDLC).

El objetivo primordial de las pruebas no es únicamente "encontrar errores", sino validar la correcta implementación de las reglas de negocio, prevenir fallos catastróficos en producción, reducir los costos de mantenimiento a largo plazo y garantizar que el producto entregado sea robusto, accesible, seguro y altamente escalable.

## 1.1. Concepto de pruebas de software

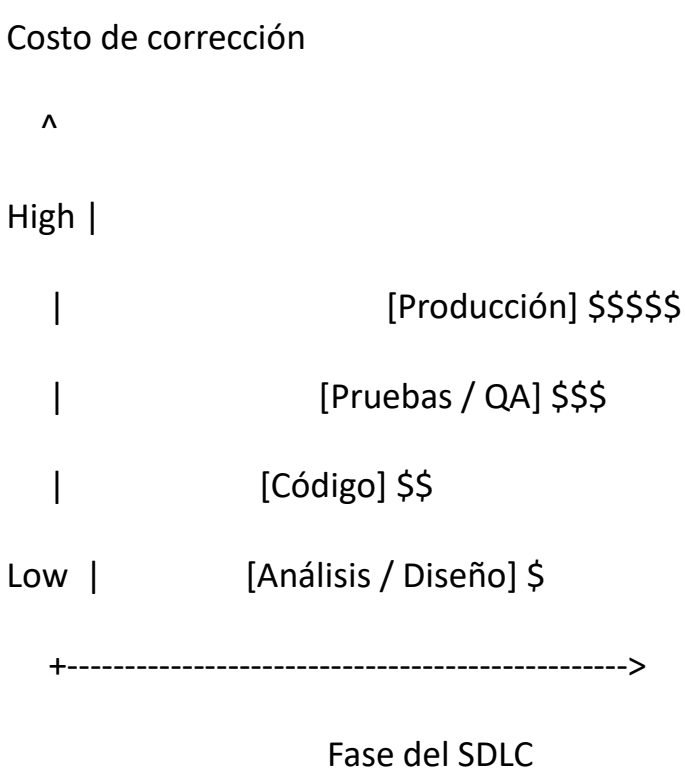
Las pruebas de software se definen formalmente como un conjunto de procesos, técnicas y actividades dinámicas y estáticas orientadas a evaluar la calidad de un artefacto de software. Esto implica la evaluación sistemática del comportamiento del sistema respecto a los requisitos funcionales y no funcionales especificados, mediante dos procesos fundamentales:

**Verificación.** Proceso estático y dinámico que responde a la pregunta ¿Se está construyendo el producto correctamente? Se enfoca en comprobar que el software cumple con las especificaciones técnicas, arquitectónicas y de diseño fijadas en las fases previas.

**Validación.** Proceso dinámico que responde a la pregunta ¿Se está construyendo el producto correcto? Se enfoca en confirmar que la aplicación satisface las necesidades reales, las expectativas y los flujos de trabajo del usuario final en un entorno operativo real.

- **La economía del error y el concepto de shift-left testing**

Uno de los fundamentos económicos más importantes en la ingeniería de software es la curva del costo de corrección de defectos (propuesta originalmente por Barry Boehm). Corregir un fallo en la fase de producción puede ser hasta 100 veces más costoso que detectarlo y solucionarlo durante la fase de análisis o diseño. El siguiente ejemplo representa de forma esquemática cómo aumenta el costo de corrección a medida que el desarrollo avanza hacia etapas posteriores del ciclo de vida del software:



Bajo este principio surge el enfoque shift-left testing (desplazar las pruebas a la izquierda), el cual promueve la ejecución de actividades de aseguramiento de calidad desde la concepción misma del requerimiento. Probar tempranamente previene la propagación de ambigüedades y errores arquitectónicos hacia las fases de codificación y despliegue.

- **Clasificación de anomalías en el software**

Para mantener una comunicación técnica rigurosa dentro del equipo multidisciplinario, es indispensable diferenciar tres conceptos clave:

- **Error.** Equivocación humana o del equipo técnico (analista, arquitecto, desarrollador) cometida durante la interpretación de un requerimiento, la lógica de un algoritmo o la escritura del código fuente.
- **Defecto (bug).** Imperfección, desviación o falla lógica presente de forma física en el código fuente, interfaz, base de datos o documentación, originada como consecuencia directa de un error humano.
- **Fallo.** Manifestación anómala o interrupción del servicio que experimenta el usuario o el sistema en tiempo de ejecución cuando el hilo de ejecución tropieza con un defecto o el sistema en tiempo de ejecución cuando el hilo de ejecución tropieza con un defecto.

## 1.2. Tipos de pruebas

La taxonomía de las pruebas de software es vasta y se estructura mediante diferentes criterios de clasificación: por su nivel de abstracción en el código, por el grado de acceso a la arquitectura interna, por su ejecutor y por la naturaleza del objetivo verificado.

- **Clasificación por nivel de ejecución**

Esta clasificación organiza las pruebas según el nivel del sistema en el que se aplican, desde la evaluación aislada de unidades de código hasta la validación integral de la solución. De acuerdo con este criterio, se distinguen los siguientes niveles de prueba:

- **Pruebas unitarias:**

Propósito: validar el correcto funcionamiento de la unidad lógica de código más pequeña de forma totalmente aislada (métodos, funciones, componentes de interfaz).

Características: son escritas y ejecutadas por los desarrolladores mediante marcos de trabajo automáticos (Jest, JUnit, PyTest, NUnit). Deben ser altamente deterministas, independientes y de ejecución ultrarápida (milisegundos).

Uso de dobles de prueba: para aislar la unidad de dependencias externas (bases de datos, APIs de terceros, red), se utilizan técnicas como mocks, stubs, spies o fakes.

- **Pruebas de Integración**

Propósito: verificar las interfaces de comunicación, la transferencia de estructuras de datos y el comportamiento conjunto entre dos o más módulos o servicios previamente probados.

Tipos comunes: integración de componentes, integración de componentes con bases de datos e integración de servicios externos (APIs REST, microservicios, colas de mensajes).

- **Pruebas de sistema**

Propósito: evaluar el comportamiento global e integral de la aplicación completa dentro de un entorno de pruebas (`staging` o QA) lo más similar posible al entorno real de producción.

Enfoque: valida flujos de trabajo completos de extremo a extremo (End-to-End – E2E), verificando tanto las reglas de negocio como la estabilidad de la infraestructura subyacente.

- **Pruebas de aceptación del usuario:**

Propósito: confirmar si el sistema está listo para su entrega comercial o despliegue en producción.

Modalidades: incluyen pruebas alfa, realizadas en un entorno controlado, y pruebas beta, aplicadas a usuarios finales externos en su entorno operativo.

Para complementar esta clasificación, las pruebas también pueden diferenciarse según el grado de conocimiento y acceso al código fuente. La siguiente tabla presenta las principales características de las pruebas de caja negra, caja blanca y caja gris:

**Tabla 1.** Clasificación según el grado de acceso al código (técnicas de caja)

| Criterio de comparación | Pruebas de caja negra (black-box) | Pruebas de caja blanca (white-box) | Pruebas de caja gris (grey-box) |
|---|---|---|---|
| Conocimiento del código. | Nulo. Se enfoca exclusivamente en la entrada y la salida (inputs / outputs). | Total. Requiere acceso pleno al código fuente, estructura y arquitectura. | Parcial. Conocimiento de modelos de datos, APIs o diagramas de arquitectura. |
| Técnicas aplicadas. | Partitioning (clases de equivalencia), valores límite, tablas de decisión. | Cobertura de sentencias, cobertura de ramas (branch coverage), cobertura de rutas. | Pruebas de integración de APIs, verificación de esquema de BD, *log* analysis. |
| Ejecutor principal. | Analistas QA, testers funcionales, usuarios finales. | Desarrolladores de software, arquitectos de código, ingenieros en automatización. | Ingenieros de automatización de pruebas, QA tech leads. |
| Nivel de aplicación habitual. | Pruebas de sistema, UAT, pruebas de usabilidad. | Pruebas unitarias, pruebas de integración de bajo nivel. | Pruebas de integración de servicios, pruebas de seguridad en APIs. |

- **Clasificación según el objetivo (funcionales vs. no funcionales)**

Esta clasificación distingue las pruebas según el aspecto del sistema que se busca evaluar su comportamiento funcional o sus atributos de calidad. A partir de este criterio, las pruebas se organizan en funcionales y no funcionales, cada una con propósitos y técnicas específicas:

**A. Pruebas funcionales**

Evalúan "qué hace" el sistema. Comprueban la lógica de negocio, manipulación de datos, autenticación, autorización y cálculos. Entre las principales pruebas funcionales se encuentran las siguientes:

- **Pruebas de regresión.** Reejecución sistemática de suites de prueba previas para verificar que las modificaciones recientes en el código, correcciones de bugs o refactorizaciones no hayan roto funcionalidades existentes.
- **Pruebas de humo.** Pruebas superficiales de respuesta rápida dirigidas a las funcionalidades más críticas para comprobar si la versión cargada es lo suficientemente estable como para iniciar un ciclo formal de pruebas.
- **Pruebas de sanidad.** Verificación focalizada en funcionalidades específicas tras un cambio menor para confirmar que el componente funciona correctamente.

**B. Pruebas no funcionales**

Evalúan "cómo lo hace" el sistema. Miden los atributos de calidad, rendimiento, resistencia, accesibilidad y seguridad. Dentro de esta categoría se encuentran las siguientes pruebas:

- **Pruebas de carga.** Someten al sistema a la demanda operativa esperada en producción (p. ej., 1000 usuarios concurrentes) para medir tiempos de respuesta y uso de CPU / memoria.
- **Pruebas de estrés.** Someten al sistema a cargas extremas que sobrepasan la capacidad máxima planificada (p. ej., 50 000 usuarios) para identificar su punto de ruptura (breaking point) y verificar si logra una recuperación limpia (graceful degradation).
- **Pruebas de volumen.** Evalúan el comportamiento de la aplicación cuando es sometida a un crecimiento masivo de datos en la base de datos.
- **Pruebas de seguridad.** Identificación y explotación simulada de vulnerabilidades de software (OWASP Top 10) para proteger la confidencialidad, integridad y disponibilidad del sistema.
- **Pruebas de usabilidad y accesibilidad.** Validan la curva de aprendizaje, la facilidad de uso y la conformidad con pautas universales de accesibilidad (WCAG 2.1 AA) para usuarios con capacidades diversas.

## 1.3. Plan de pruebas

El plan de pruebas (test plan) es el artefacto formal y estratégico que define los objetivos, el alcance, los recursos, las herramientas, los entornos, el cronograma y la gestión de riesgos asociados con el proceso de aseguramiento de calidad. Es un documento dinámico regido por estándares internacionales como ISO/IEC/IEEE 29119-3.

Un plan de pruebas profesional integra diferentes componentes que permiten delimitar el proceso, establecer las condiciones para su ejecución y determinar los

recursos y entregables necesarios. Entre sus principales componentes estructurales se encuentran los siguientes:

- **Identificador del documento y control de cambios.** Permite mantener la trazabilidad de las versiones del plan de pruebas mediante el registro de modificaciones, fechas, autores, responsables y aprobadores. Este componente facilita el seguimiento de la evolución del documento y permite identificar qué cambios se realizaron, quién los efectuó y cuál es la versión vigente durante el proceso de pruebas.
- **Introducción y alcance (scope).** Delimita qué elementos serán objeto de prueba y cuáles quedan excluidos. El alcance (in-scope) comprende los módulos, navegadores, dispositivos, sistemas operativos y funcionalidades que serán probados. Por su parte, los elementos fuera del alcance (out-of-scope) corresponden a componentes, integraciones de terceros o plataformas que se excluyen explícitamente del proceso para evitar desviaciones.
- **Estrategia y niveles de prueba.** Describe el enfoque metodológico, los niveles de prueba, las herramientas que se utilizarán —como Postman, Selenium, JMeter, Cypress o Appium— y el nivel de automatización proyectado para orientar la ejecución del proceso.
- **Criterios de entrada, suspensión y salida.** Establecen las condiciones que regulan el inicio, la posible interrupción y la finalización de las pruebas. Los criterios de entrada (entry criteria) determinan los requisitos previos para comenzar; los criterios de suspensión (suspension criteria) establecen las circunstancias que obligan a detener temporalmente el

proceso; y los criterios de salida (`exit criteria`) especifican las metas que deben alcanzarse para dar por concluida la fase de pruebas.

- **Entornos y datos de prueba.** Comprenden la especificación del hardware, la topología de red, los certificados SSL, las versiones del motor de base de datos y las estrategias para enmascarar o generar datos sintéticos, considerando las regulaciones aplicables de protección de datos personales.
- **Entregables de pruebas.** Incluyen los documentos y reportes generados durante el proceso, como casos de prueba, reportes de ejecución, matrices de trazabilidad de requisitos y reportes de defectos gestionados mediante herramientas como Jira o Azure DevOps.

La planificación también debe considerar los riesgos que pueden afectar la ejecución de las pruebas y las acciones previstas para responder ante ellos. La siguiente tabla ejemplifica la relación entre el riesgo identificado, su impacto, su probabilidad y el correspondiente plan de mitigación o contingencia:

**Tabla 2.** Matriz de gestión de riesgos y contingencias

| Riesgo identificado | Impacto | Probabilidad | Plan de mitigación / contingencia |
|---|---|---|---|
| Retraso en la entrega del entorno de pruebas. | Alto. | Media. | Utilizar contenedores `docker` estandarizados para desplegar entornos locales en máquinas de desarrollo. |

| Riesgo identificado | Impacto | Probabilidad | Plan de mitigación / contingencia |
|---|---|---|---|
| Inestabilidad en la API de pagos del proveedor. | Crítico. | Alta. | Implementar un servidor mock de pruebas que simule las respuestas de la pasarela de pagos. |
| Rotación o falta de personal capacitado en automatización. | Medio. | Media. | Diseñar *scripts* de automatización modulares bajo patrones de diseño como Page Object Model (POM) para facilitar su mantenimiento. |

## 1.4. Criterios de aceptación

Los criterios de aceptación son el conjunto formal, explícito y detallado de condiciones, límites y reglas de negocio que una historia de usuario o requerimiento debe satisfacer plenamente para que el product owner o cliente la apruebe y la acepte como entregada.

Su importancia operativa tiene en cuenta lo siguiente:

- Eliminan la ambigüedad: transforman intenciones generales del usuario en especificaciones técnicas verificables.
- Definen los límites del alcance: previenen el crecimiento descontrolado del alcance (scope creep).
- Guían el diseño de pruebas: sirven como la fuente de entrada primaria a partir de la cual el analista QA redacta los casos de prueba.

- **Redacción basada en BDD (Behavior-Driven Development)**

El enfoque BDD promueve el uso de un lenguaje omnipresente y comprensible tanto para perfiles técnicos como de negocio, utilizando la sintaxis formal Gherkin:

  - **Dado que (given).** Establece el estado inicial del sistema, el contexto o las precondiciones necesarias.
  - **Cuando (when).** Especifica la acción principal, el evento o el gatillo ejecutado por el usuario o un proceso automático.
  - **Entonces (then).** Detalla el resultado esperado, la reacción del sistema o las poscondiciones observables.
  - **Y / pero (and / but).** Conectores lógicos para extender precondiciones, acciones o verificaciones adicionales.

- **Ejemplo práctico de historia de usuario y criterios de aceptación**

El siguiente ejemplo muestra cómo una necesidad expresada mediante una historia de usuario puede transformarse en criterios de aceptación verificables. Primero se plantea el requerimiento desde la perspectiva del usuario y, posteriormente, se estructura mediante la sintaxis Gherkin, definiendo escenarios que permiten comprobar el comportamiento esperado del sistema:

  - **Historia de usuario**
    - Como cliente registrado en la tienda virtual.
    - Quiero aplicar un código de descuento promocional en la pantalla de pago.
    - Para obtener un beneficio económico en el total de mi compra.

A partir de esta historia de usuario, los criterios de aceptación se codifican mediante escenarios en Gherkin, utilizando las expresiones Dado que, Cuando y Entonces para representar las precondiciones, acciones y resultados esperados de la siguiente manera:

**Escenario 1:** Aplicación exitosa de código de descuento válido

Dado que el usuario tiene un carrito con productos por un total de $100.000 COP

Y el usuario se encuentra en la pantalla de verificación de pago

Cuando ingresa el código promocional "VERANO2026" en el campo de texto

Y presiona el botón "Aplicar Código"

Entonces el sistema debe validar el código contra la base de datos

Y aplicar un descuento del 10 % ($10.000 COP) sobre el total de la compra

Y actualizar el monto final a pagar a $90.000 COP en menos de 2 segundos.

**Escenario 2:** Intento de aplicación de código promocional vencido

Dado que el usuario se encuentra en la pantalla de verificación de pago

Cuando ingresa el código promocional "NAVIDAD2025"

Y presiona el botón "Aplicar Código"

Entonces el sistema no debe aplicar ningún descuento al total acumulado

Y debe desplegar un mensaje de advertencia visible en color rojo: "El código promocional ingresado ha expirado".

## 1.5. Definition of Ready y Definition of Done

En el marco de las metodologías ágiles como Scrum y Kanban, el DoR (Definition of Ready) y el DoD (Definition of Done) funcionan como acuerdos de calidad contractuales dentro del equipo. Funcionan como compuertas de inspección (quality gates) que regulan la entrada y salida de las historias de usuario a lo largo de las iteraciones (Sprints).

El siguiente esquema representa el flujo de una historia de usuario y permite identificar los momentos en los que se aplican el DoR y el DoD, desde su incorporación al backlog hasta la obtención de un incremento desplegable:

[ Historias en backlog ] —► [ DoR ] —► [ Desarrollo / Sprint ] —► [ DoD ] —► [ Incremento desplegable ]

- **Definition of Ready (DoR) – definición de listo**

Es la lista de comprobación que evalúa si una historia de usuario está adecuadamente madura, especificada y refinada antes de ser aceptada dentro del compromiso de desarrollo de un Sprint. Su objetivo es evitar que los desarrolladores comiencen a codificar sobre requerimientos incompletos o ambiguos.

Una historia cumple el DoR si satisface el estándar INVEST, acrónimo que reúne seis características fundamentales:

- **Independent (independiente).** Se puede desarrollar e implementar sin bloqueos o dependencias directas con otras historias paralelas.

- **Negotiable (negociable).** Los detalles son discutidos y acordados colaborativamente entre el product owner y el equipo.
- **Valuable (valiosa).** Aporta un valor claro y medible al usuario o al negocio.
- **Estimable (estimable).** El equipo comprende el alcance técnico para poder otorgarle un tamaño (puntos de historia u horas).
- **Small (pequeña).** Tiene el tamaño óptimo para ser completada holgadamente dentro del marco del sprint.
- **Testable (comprobable).** Posee criterios de aceptación claros que permiten construir pruebas para verificarla.

- **Definition of Done (DoD) – definición de hecho**

Es la lista compartida de criterios técnicos, operacionales y de calidad que debe cumplir obligatoriamente todo elemento del backlog antes de darlo por terminado. Garantiza que el producto resultante sea un incremento funcional de software de alta calidad, con la deuda técnica controlada y potencialmente desplegable a producción. A partir de estas características, es posible diferenciar el DoR y el DoD según su momento de aplicación, objetivo, responsables, consecuencias de incumplimiento y criterios de verificación. La siguiente tabla presenta esta comparación:

**Tabla 3.** Cuadro comparativo integral entre DoR y DoD

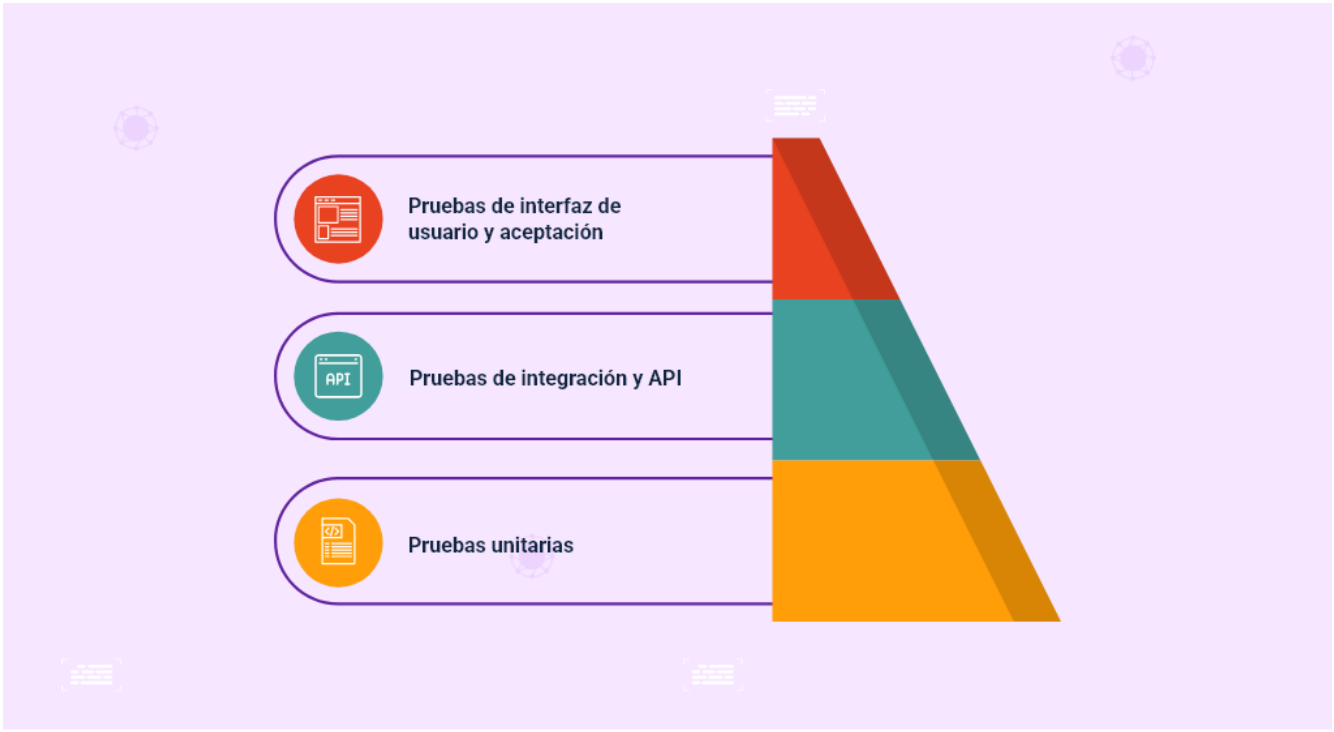
| Dimensión | Definition of Ready (DoR) | Definition of Done (DoD) |
|---|---|---|
| Punto de aplicación. | Fase de entrada: antes de planificar e iniciar la iteración (Sprint planning). | Fase de salida: al finalizar el ciclo de desarrollo y pruebas de la tarea. |

| Dimensión | Definition of Ready (DoR) | Definition of Done (DoD) |
|---|---|---|
| Objetivo central. | Garantizar la claridad, viabilidad y comprensión del requerimiento. | Garantizar la excelencia técnica, la calidad y la ausencia de deuda técnica. |
| Liderazgo principal. | Product owner, analistas de negocio y arquitecto. | Equipo de desarrolladores, ingenieros QA y DevOps. |
| Consecuencia de incumplir. | La historia no ingresa al sprint; se devuelve a refinamiento. | La historia no se da por terminada ni suma a la velocidad del Sprint. |
| Lista de chequeo típica. | • Historia redactada bajo el formato estándar.<br>• Criterios de aceptación (Gherkin) aprobados por QA.<br>• Mapeo de UX / UI disponible (wireframes / mockups).<br>• Tarea estimada por el equipo técnico.<br>• Dependencias técnicas resueltas. | • Código fuente completamente escrito y refactorizado.<br>• Análisis estático de código sin alertas críticas (SonarQube).<br>• Pruebas unitarias ejecutadas con cobertura mínima del 80 %.<br>• Pruebas de integración y regresión aprobadas. |

| Dimensión | Definition of Ready (DoR) | Definition of Done (DoD) |
|---|---|---|
| | | • Revisión de código (code review) aprobada por 2 pares.<br>• Documentación técnica y de usuario actualizada.<br>• Despliegue exitoso en el entorno de pruebas (staging). |

Además de los criterios que orientan la preparación y finalización de las historias de usuario, la estrategia de calidad también requiere organizar las pruebas según su nivel de alcance. La siguiente figura representa esta distribución mediante la pirámide de automatización de pruebas:

**Figura 1.** Pirámide de automatización de pruebas e interacción de capas de prueba


Pruebas de interfaz de usuario y aceptación
API
Pruebas de integración y API
Pruebas unitarias

## 2. Prototipado de aplicaciones

El prototipado de aplicaciones es una disciplina transversal en el diseño de experiencia de usuario (UX) e interfaz de usuario (UI). Consiste en la creación de representaciones visuales, estructurales e interactivas de una solución digital antes de iniciar la fase de codificación formal. Prototipar permite validar hipótesis de diseño, evaluar la usabilidad con usuarios reales, alinear las expectativas de los interesados (stakeholders) y mitigar los riesgos de reproceso en las etapas avanzadas del desarrollo. Este proceso avanza desde la definición inicial de la idea o requerimiento hasta la construcción de representaciones cada vez más detalladas e interactivas, como se muestra en el siguiente esquema:

[ Idea / requerimiento ] —► [ Sketch ] —► [ Wireframe ] —► [ Mockup ] —► [ Wireflow / prototipo ]

### 2.1. Concepto de prototipo

Un prototipo es un modelo preliminar o maqueta interactiva de un sistema informático que simula la apariencia, la arquitectura de información y la dinámica de navegación de la aplicación final.

El prototipado se fundamenta en tres principios principales: la iteratividad, que permite probar, ajustar y refinar la interfaz de manera ágil y económica antes de escribir código; la comunicación clara, al proporcionar un lenguaje común entre diseñadores, desarrolladores, analistas y clientes; y la validación temprana, que facilita la realización de pruebas de usabilidad para identificar oportunamente problemas de navegación o jerarquía visual.

- **Clasificación según el nivel de fidelidad**

La fidelidad de un prototipo hace referencia al grado de precisión gráfica, funcional y de detalle que posee en comparación con el producto final. De acuerdo con este nivel de fidelidad, los prototipos pueden clasificarse en baja, media y alta fidelidad. La siguiente tabla compara sus principales características, herramientas, niveles de interactividad y momentos de uso:

**Tabla 4.** Clasificación y características de los prototipos según su nivel de fidelidad

| Criterio | Baja fidelidad (low-fidelity) | Media fidelidad (medium-fidelity) | Alta fidelidad (high-fidelity) |
|---|---|---|---|
| Representación gráfica. | Esquemática, monocromática o a mano alzada. | Estructurada, escala de grises, tipografías estándar. | Detallada, paleta de colores final, imágenes y tipografía real. |
| Interactividad. | Nula o simulación manual en papel. | Navegación básica entre pantallas mediante clics. | Animaciones, transiciones, microinteracciones y respuestas a eventos. |
| Herramientas de creación. | Papel, lápiz, tableros, balsamiq. | Figma, Sketch, Adobe XD, Axure RP. | Figma, ProtoPie, InVision, código frontend (HTML / CSS). |

| Criterio | Baja fidelidad (low-fidelity) | Media fidelidad (medium-fidelity) | Alta fidelidad (high-fidelity) |
|---|---|---|---|
| Costo y tiempo. | Ultrabajo y de ejecución inmediata. | Moderado, requiere estructuración de componentes. | Alto, requiere trabajo detallado de diseño UX / UI. |
| Momento de uso. | Fases iniciales de ideación y brainstorming. | Definición de arquitectura y flujos de pantalla. | Validación con clientes y entrega a desarrolladores (handoff). |

## 2.2. Sketch y mockups

Dentro del flujo de trabajo de diseño UI, los Sketches y los mockups se ubican en diferentes niveles de fidelidad visual y cumplen funciones complementarias en el proceso de diseño. A continuación, se explican las características y el propósito de cada uno dentro del desarrollo de una interfaz:

- **Sketch (boceto inicial)**

El sketch es una representación gráfica inicial y de baja fidelidad que permite explorar rápidamente ideas y alternativas de diseño. Puede realizarse a mano alzada con lápiz y papel o mediante herramientas digitales, y se utiliza principalmente para definir la organización general de la interfaz antes de incorporar detalles visuales.

Características principales:

- Creación e iteración rápida.
- Baja fidelidad visual.
- Representación de la distribución de los componentes.
- Facilidad para explorar diferentes ideas de diseño.

- **Mockup (maqueta de alta fidelidad)**

El mockup es una representación estática y de alta fidelidad que permite visualizar con mayor precisión la apariencia que tendrá la interfaz antes de su implementación. Incorpora los principales elementos gráficos y facilita la definición y validación de la propuesta visual del producto.

Características principales:

- Alta fidelidad visual.
- Definición de colores y tipografías.
- Incorporación de iconografía y recursos multimedia.
- Representación detallada de los componentes UI.

## 2.3. Wireframe

Un wireframe (o esquema de página) es un plano o estructura de baja o media fidelidad que define la disposición espacial de los elementos en una pantalla, la jerarquía de la información y los puntos de interacción, sin incluir elementos estéticos como color o imágenes definitivas.

El siguiente ejemplo representa la estructura básica de un wireframe y muestra cómo pueden distribuirse los principales elementos de una interfaz antes de incorporar los detalles visuales definitivos:

```
+-----------------------------------------+
| [ LOGO ]      [ Inicio ] [ Servicios ] [ Contacto ] |
+-----------------------------------------+
|                                         |
|   [ BANNER PRINCIPAL / TITULAR ]        |
|   Texto descriptivo breve...            |
|   [ BOTÓN CTA ]                         |
|                                         |
+-----------------------------------------+
| [ Card 1 ]     | [ Card 2 ]   | [ Card 3 ] |
| (X) Imagen     | (X) Imagen   | (X) Imagen |
| Texto...       | Texto...     | Texto...   |
+-----------------------------------------+
```

El wireframe cumple diferentes propósitos durante la planificación y estructuración de una interfaz, ya que permite definir aspectos funcionales antes de incorporar los elementos visuales definitivos. Entre los principales se encuentran:

- **Centrarse en la usabilidad.** Al omitir el color y los detalles gráficos, el equipo se concentra exclusivamente en la funcionalidad, la navegación y la claridad del contenido.
- **Definir la arquitectura de información.** Establece qué elementos son prioritarios y cómo se agrupan en la pantalla.
- **Facilitar la maquetación.** Sirve como mapa conceptual directo para que los desarrolladores frontend entiendan la estructura HTML necesaria.

## 2.4. Wireflow

Un wireflow es la combinación estructural de varios wireframes (o mockups) conectados mediante líneas que representan el flujo de interacción. Su objetivo es mapear visualmente el recorrido del usuario (user journey) a través de la aplicación ante diferentes acciones o eventos.

Para representar de manera clara la navegación y las posibles rutas de interacción, un wireflow integra diferentes elementos que permiten relacionar las pantallas con las acciones del usuario. Entre sus principales componentes se encuentran:

- Pantallas o nodos que son representaciones de interfaces mediante wireframes o mockups.
- Conectores o flechas de flujo que indican la dirección del desplazamiento según la interacción del usuario.
- Gatillos (triggers) que corresponden a botones, enlaces, gestos táctiles o formularios que desencadenan el cambio de estado o pantalla.

- Condicionales de decisión que representan puntos del flujo donde el sistema evalúa una regla; por ejemplo: ¿Usuario autenticado? Sí / No.

A partir de los conceptos abordados, es posible establecer las principales diferencias entre los artefactos utilizados en el proceso de prototipado. La siguiente tabla compara cada uno según su nivel de fidelidad, grado de interactividad y propósito principal:

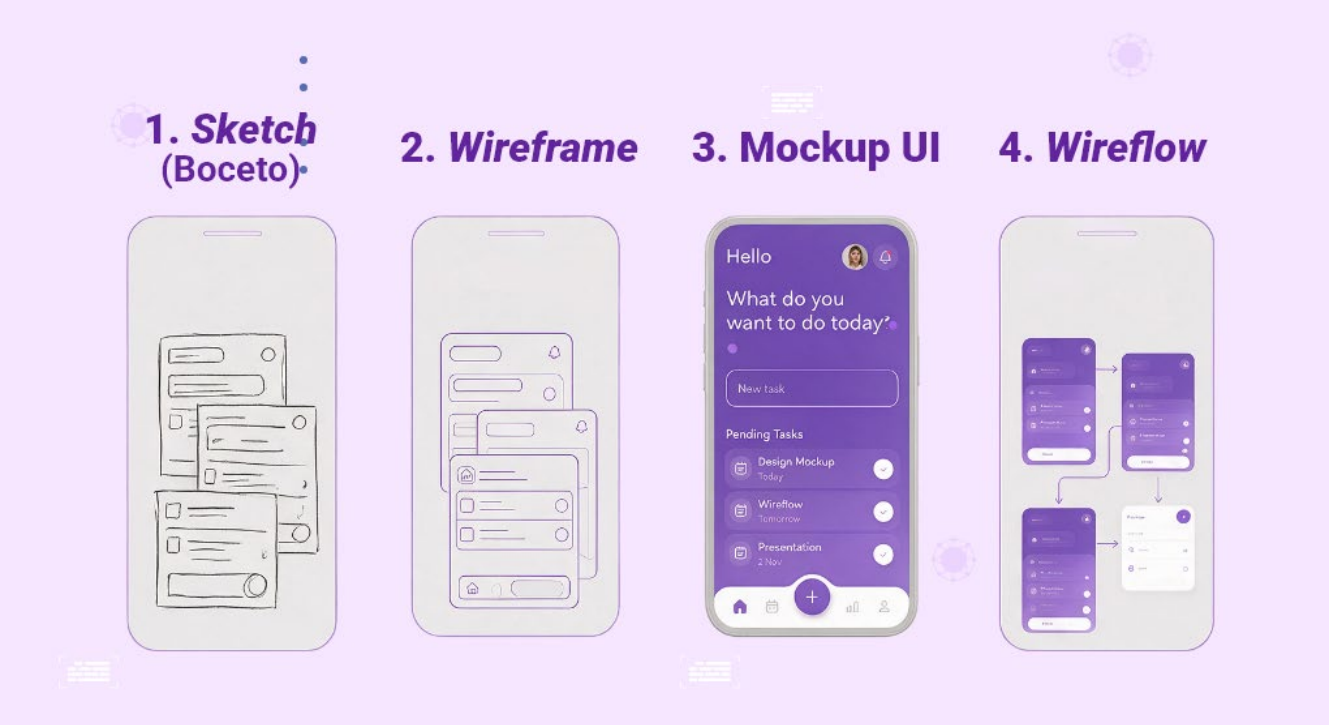
**Tabla 5.** Matriz comparativa de los artefactos del proceso de prototipado

| Artefacto | Nivel de fidelidad | ¿Es dinámico / interactivo? | Propósito principal |
|---|---|---|---|
| Sketch. | Baja. | No (estático). | Generar e iterar ideas de forma rápida a mano alzada. |
| Wireframe. | Baja / media. | No (estático). | Organizar la estructura, jerarquía y distribución del contenido. |
| Mockup. | Alta. | No (estático). | Definir el diseño visual, paleta de colores y componentes estéticos. |
| Wireflow. | Media / alta. | Sí (diagrama de flujo). | Mapear la navegación, rutas e interacciones del usuario entre pantallas. |

Esta comparación permite analizar cómo los artefactos evolucionan desde representaciones iniciales de baja fidelidad hasta recursos con mayor nivel de detalle e

interactividad. La siguiente figura representa gráficamente esta evolución dentro del proceso de prototipado:

**Figura 2.** Evolución de los artefactos de diseño e interactividad en el proceso de prototipado



# 3. Desarrollo de software

El desarrollo de software es una disciplina de la ingeniería que combina principios teóricos, metodologías de trabajo, herramientas tecnológicas y modelos de gestión para diseñar, codificar, probar, desplegar y mantener soluciones informáticas. La correcta selección del ciclo de vida y la metodología de desarrollo determina la eficiencia operativa del equipo, la calidad del producto entregado y la capacidad del proyecto para adaptarse a las dinámicas cambiantes del negocio.

## 3.1. Concepto de ciclo de vida del software

El ciclo de vida del desarrollo de software (SDLC) es la estructura que organiza las fases, actividades y entregables necesarios para llevar un producto de software desde su concepción inicial hasta su retiro. Este ciclo proporciona un marco para planificar y controlar el desarrollo, facilitando la coordinación de las actividades y la toma de decisiones durante las distintas etapas del proyecto.

Entre sus principales objetivos se encuentran estandarizar los procesos, proporcionando un marco de trabajo común para el equipo; garantizar la calidad, mediante actividades de verificación e inspección durante el desarrollo; y gestionar los riesgos y costos, considerando la asignación de recursos, tiempos y costos operativos en cada etapa.

- **Clasificación de los modelos de ciclo de vida**

Los modelos de ciclo de vida pueden organizarse de acuerdo con la forma en que estructuran el desarrollo y responden a los cambios en los requisitos. De manera general, se distinguen los siguientes enfoques:

- **Modelos predictivos (tradicionales o secuenciales).** Se basan en una planificación inicial detallada y en una definición anticipada de los requisitos, con una secuencia de desarrollo previamente establecida. Entre sus ejemplos se encuentran el modelo en cascada (waterfall) y el modelo en V.
- **Modelos adaptativos (iterativos e incrementales).** Organizan el desarrollo mediante ciclos sucesivos que permiten construir y entregar progresivamente el producto, incorporar retroalimentación y responder a cambios durante el proyecto. En este enfoque pueden emplearse marcos y métodos como Scrum, Kanban y Extreme Programming (XP).

## 3.2. Metodologías de desarrollo

Las metodologías de desarrollo representan la puesta en práctica de los modelos de ciclo de vida. Definen los roles, artefactos, actividades y reglas de interacción que orientan la organización del trabajo y permiten coordinar las diferentes etapas del desarrollo de software. Según la forma en que se planifica, ejecuta y adapta el trabajo, pueden emplearse diferentes enfoques. A continuación, se presentan la metodología en cascada y los marcos ágiles:

### a. Metodología en cascada (waterfall)

La metodología en cascada sigue un enfoque secuencial en el que el desarrollo avanza mediante fases previamente definidas, como análisis de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento. En su formulación tradicional,

cada fase se desarrolla después de completar la anterior, lo que facilita la planificación y el seguimiento del proyecto.

Este enfoque se caracteriza por una planificación detallada, una documentación amplia y una definición anticipada de los requisitos, tiempos y costos. Estas características favorecen el control del proyecto cuando los requisitos son estables; sin embargo, su estructura ofrece menor flexibilidad ante cambios y puede generar reprocesos cuando se identifican nuevas necesidades o problemas en etapas avanzadas.

**b. Marcos ágiles (agile frameworks)**

Los enfoques ágiles promueven el desarrollo iterativo e incremental, la colaboración continua y la capacidad de adaptación frente a cambios en los requisitos. El trabajo se organiza de manera que sea posible obtener retroalimentación frecuente y realizar ajustes durante el desarrollo, en lugar de concentrar la validación y la entrega únicamente al final del proyecto.

Dentro del enfoque ágil existen diferentes marcos y métodos para organizar y gestionar el trabajo. Entre los más utilizados se encuentran:

- **Scrum.** Es un marco ágil que organiza el trabajo mediante ciclos de duración definida denominados sprints. Establece responsabilidades, eventos y artefactos que permiten planificar el trabajo, desarrollar incrementos del producto, revisar los resultados obtenidos e incorporar ajustes de manera periódica. De esta forma, favorece la inspección, la adaptación y la entrega progresiva de valor.

- **Kanban.** Es un método de gestión visual orientado a optimizar el flujo de trabajo. Permite visualizar las actividades y su estado, controlar la cantidad de trabajo en proceso (Work in Progress – WIP) e identificar posibles cuellos de botella. A diferencia del trabajo organizado en iteraciones fijas, Kanban favorece un flujo continuo, en el que las tareas avanzan conforme existe capacidad disponible.

Las diferencias entre los enfoques tradicionales y ágiles pueden analizarse en aspectos como la gestión de los requisitos, las entregas de software, la respuesta al cambio, la medición del progreso y la participación del cliente. La siguiente tabla compara estos elementos:

**Tabla 6.** Matriz comparativa entre la metodología tradicional y el marco ágil

| **Criterio de comparación** | **Metodología en cascada (waterfall)** | **Marco ágil (Scrum)** |
|---|---|---|
| Gestión de requisitos. | Definidos y detallados al inicio del proyecto, con cambios gestionados formalmente. | Dinámicos, evolucionan mediante el product backlog. |
| Entregas de software. | La entrega del producto completo se realiza generalmente al finalizar el proyecto. | Entregas incrementales al finalizar cada sprint. |
| Respuesta al cambio. | Los cambios requieren evaluación y gestión formal | Flexible, permite incorporar cambios para aportar mayor valor. |

| Criterio de comparación | Metodología en cascada (waterfall) | Marco ágil (Scrum) |
|---|---|---|
| | debido a la planificación secuencial. | |
| Medición del progreso. | Basada en el cumplimiento de hitos y documentos de la fase. | Basada en el avance hacia los objetivos y en incrementos utilizables del producto. |
| Participación del cliente. | Alta al inicio (requisitos) y al final (pruebas). | Continua y constante a lo largo de cada iteración. |

## 3.3. Ejecución de la metodología

La ejecución metodológica requiere la adopción de herramientas, artefactos y mecanismos de gestión visual que permitan organizar y coordinar el trabajo cotidiano del equipo. En los enfoques ágiles, estos elementos facilitan la priorización de las necesidades, la planificación de las actividades y el seguimiento del avance durante el desarrollo.

Entre los principales artefactos utilizados se encuentra el product backlog, que reúne y prioriza las características, historias de usuario, correcciones y demás necesidades relacionadas con el producto. Su contenido puede evolucionar a medida que cambian las prioridades y se obtiene nueva información sobre el producto.

A partir del product backlog se organiza el sprint backlog, que contiene el trabajo seleccionado para desarrollar durante un sprint. Este artefacto permite hacer

visible la planificación del equipo y orientar las actividades necesarias para alcanzar el objetivo establecido para la iteración.

Como resultado del trabajo realizado se obtiene el incremento, entendido como el conjunto integrado de elementos completados que cumplen con los criterios de calidad establecidos en la Definition of Done (DoD). De esta manera, los artefactos permiten conectar la planificación del producto con la ejecución y los resultados obtenidos durante el desarrollo.

- **Gestión visual del flujo de trabajo**

Además de los artefactos de planificación y seguimiento, los equipos pueden utilizar tableros visuales para representar el estado de las actividades y facilitar el control del flujo de trabajo. Estos tableros, físicos o implementados mediante herramientas como Jira o Azure DevOps, organizan las tareas en columnas que reflejan las diferentes etapas por las que avanzan durante su ejecución.

El siguiente ejemplo representa, mediante una estructura de código, un tablero básico de seguimiento en el que las tareas se distribuyen según su estado:

```
+-------------------------------------------------------------+
| POR HACER (To Do) | EN PROCESO (WIP) | EN PRUEBAS (QA) | HECHO (Done) |
+-------------------------------------------------------------+
| [ Tarea A ]      | [ Tarea B ]     | [ Tarea C ]     | [ Tarea D ] |
| [ Tarea E ]      |                 |                 |             |
+-------------------------------------------------------------+
```

## 3.4. Fases del desarrollo de software

El desarrollo de software comprende un conjunto de fases que permiten transformar una necesidad o problema en una solución tecnológica funcional, validada y mantenible. Estas fases abarcan desde la identificación y análisis de los requisitos hasta la puesta en producción y evolución del sistema. Su organización y ejecución pueden variar según la metodología adoptada: en enfoques secuenciales suelen desarrollarse de manera progresiva, mientras que en enfoques iterativos o ágiles pueden repetirse y articularse a lo largo de diferentes ciclos de desarrollo.

A continuación, se explican las principales fases que intervienen en este proceso:

**A. Análisis de requisitos y factibilidad**

En esta fase se identifican y analizan las necesidades del negocio y de los usuarios mediante técnicas como entrevistas, talleres y levantamiento de información. También se estudia la viabilidad técnica, operativa, legal y financiera del proyecto con el propósito de determinar si la solución puede desarrollarse bajo las condiciones establecidas. Como parte de este proceso, los requisitos pueden documentarse formalmente mediante una Especificación de Requisitos de Software (SRS).

**B. Diseño de arquitectura y sistema**

Esta fase establece cómo estará estructurada la solución antes de su implementación. Comprende la definición de la arquitectura de software, que puede adoptar enfoques monolíticos, de microservicios u orientados a eventos, así como el diseño de los modelos de datos, diagramas de clases y contratos de API. También contempla el diseño de interfaces y prototipos, mediante recursos como wireframes y mockups, para representar anticipadamente la estructura y apariencia de la solución.

### C. Codificación e implementación

Durante esta fase, el diseño se transforma en código fuente mediante lenguajes, tecnologías y estándares definidos para el proyecto. Los desarrolladores pueden aplicar patrones de diseño, como MVC, Repository o Factory, para favorecer la organización y mantenibilidad del código. Asimismo, se utilizan sistemas de control de versiones como Git y estrategias de ramificación como GitFlow o Trunk-Based Development para gestionar los cambios y facilitar el trabajo colaborativo.

### D. Pruebas y aseguramiento de la calidad

Esta fase busca verificar y validar que el software cumpla los requisitos establecidos y presente un comportamiento adecuado. Para ello, se realizan diferentes tipos de pruebas, como unitarias, de integración, funcionales, de rendimiento y de seguridad. Los defectos identificados se registran, corrigen y vuelven a evaluar con el propósito de comprobar su resolución y reducir el riesgo de fallos antes de la liberación del producto.

### E. Despliegue y puesta en producción

Una vez que el producto alcanza las condiciones establecidas para su liberación, se prepara y despliega en la infraestructura correspondiente, que puede estar alojada en servidores propios o en plataformas de nube como AWS, Azure o GCP. Este proceso puede apoyarse en mecanismos de integración y despliegue continuos (CI / CD) y herramientas como Jenkins, GitHub Actions o GitLab CI, que permiten automatizar diferentes actividades de construcción, prueba y despliegue.

### F. Mantenimiento y evolución

Después de la puesta en producción, el software continúa siendo supervisado y actualizado para conservar su funcionamiento y responder a nuevas necesidades. El mantenimiento correctivo atiende fallos o incidentes detectados durante la operación; el mantenimiento evolutivo incorpora nuevas funcionalidades o mejoras de acuerdo con las necesidades del producto o del negocio; y el mantenimiento adaptativo realiza modificaciones para conservar la compatibilidad frente a cambios en sistemas operativos, plataformas, dependencias o tecnologías del entorno.

Aunque las fases se presentan de manera secuencial para facilitar su comprensión, su ejecución depende de la metodología adoptada. En los enfoques tradicionales suelen desarrollarse de forma progresiva, mientras que en los enfoques iterativos y ágiles pueden repetirse y articularse a lo largo de diferentes ciclos de desarrollo.

# 4. Contenido web

El contenido web abarca todos los artefactos de información, medios interactivos y estructuras de datos que se transmiten a través de los protocolos HTTP y HTTPS para ser procesados e interpretados por navegadores web o aplicaciones cliente. En el contexto de la ingeniería de software y el diseño de experiencias digitales, la gestión del contenido web no se limita a la simple redacción de textos o inserción de imágenes; requiere una estrategia integral de estructuración semántica, optimización de recursos, licenciamiento legal y administración eficiente mediante bases de datos relacionales y no relacionales.

## 4.1. Características y distribución del contenido web

El contenido web moderno debe diseñarse considerando principios de adaptabilidad, accesibilidad, indexabilidad y velocidad de carga. La forma en que se distribuye la información condiciona directamente la usabilidad del sitio y su posicionamiento en motores de búsqueda (Search Engine Optimization – SEO). Estos principios se concretan en características que orientan la estructuración, presentación e interacción del contenido dentro de una página o aplicación web.

Entre las principales características que favorecen una organización adecuada del contenido se encuentran las siguientes:

- **Estructuración semántica.** Uso de etiquetas HTML5 (<header>, <nav>, <main>, <article>, <section>, <footer>) que otorgan significado contextual al contenido, facilitando su lectura por parte de tecnologías de asistencia (lectores de pantalla) y crawlers de búsqueda.

- **Escaneabilidad.** Organización del texto mediante jerarquías claras de encabezados (<h1> a <h6>), párrafos breves, listas con viñetas y resaltados en negrita para permitir la lectura rápida.
- **Hipertextualidad e interactividad.** Capacidad de interconectar nodos de información mediante enlaces y permitir la participación activa del usuario mediante formularios y componentes reactivos.
- **Adaptabilidad.** Disposición fluida del contenido que se ajusta automáticamente a diferentes tamaños de pantalla (móviles, tabletas, monitores 4K) mediante Media Queries CSS y diagramación flexible.

Además de su organización y presentación, el contenido debe distribuirse eficientemente desde la infraestructura que lo aloja hasta el navegador del usuario. El siguiente esquema representa de manera simplificada este proceso mediante el uso de una red de distribución de contenido (Content Delivery Networks – CDN):

[ Servidor origen ] —► [ Red de distribución de contenido (CDN) ] —► [ Navegador usuario ]

├— Caching en Edge Servers

└— Reducción de latencia global

- **Arquitectura de distribución de contenido**

Para garantizar tiempos de respuesta óptimos a nivel global, la distribución de archivos estáticos (imágenes, hojas de estilo CSS, scripts JS, videos) se apoya en redes de distribución de contenido (CDN). Una CDN replica los recursos en una red geográfica de servidores en el borde (Edge Servers), entregando el contenido desde el nodo más cercano al usuario final.

En este modelo, el servidor de origen conserva los recursos principales, mientras que los nodos distribuidos pueden almacenar copias temporales mediante mecanismos de caché. Cuando un usuario solicita un recurso disponible en un nodo cercano, este puede entregarse directamente desde dicho nodo, reduciendo la necesidad de realizar nuevas solicitudes al servidor de origen. Este funcionamiento contribuye a disminuir la latencia, reducir la carga sobre la infraestructura principal y mejorar la velocidad de entrega del contenido.

## 4.2. Formatos y tipos de archivo

La selección de los formatos de archivo adecuados impacta directamente en el rendimiento de red, el ancho de banda consumido y la fidelidad visual de la aplicación. Cada tipo de recurso web requiere un formato acorde con su función, ya sea para estructurar contenido, definir estilos, incorporar interactividad, representar elementos gráficos o facilitar el intercambio de datos.

La elección adecuada permite equilibrar compatibilidad, calidad y eficiencia, considerando las características del recurso y el contexto en el que será utilizado. La siguiente tabla presenta los principales formatos empleados en contenido web, sus características y algunos de sus casos de uso recomendados:

**Tabla 7.** Clasificación de formatos de archivo estándar para contenido web

| Categoría | Formato / extensión | Características principales | Caso de Uso recomendado |
|---|---|---|---|
| Texto / estructura. | .html / .htm | Lenguaje de marcado de hipertexto estandarizado. | Estructura base de páginas web. |
| Estilos. | .css | Hojas de estilo en cascada para la capa visual. | Diagramación, diseño y animación. |
| Lógica del lado del cliente (frontend). | .js / .mjs | Lenguaje de programación interpretado en cliente. | Interactividad y dinamismo UI. |
| Imagen vectorial. | .svg | Gráficos basados en XML, escalables sin pérdida. | Logotipos, íconos, diagramas e ilustraciones. |
| Imagen ráster. | .webp / .avif | Formatos modernos de alta compresión con o sin pérdida. | Fotografías, *banners* e imágenes complejas. |
| Imagen en formato tradicional. | .jpg / .png / .gif | Formatos tradicionales ampliamente compatibles. | Compatibilidad con navegadores antiguos. |
| Tipografía web. | .woff2 / .woff | Formato comprimido optimizado para la web. | Fuentes tipográficas personalizadas. |

| Categoría | Formato / extensión | Características principales | Caso de Uso recomendado |
|---|---|---|---|
| Datos / intercambio. | .json / .xml | Formatos livianos para estructuración e intercambio de datos. | Consumo de APIs REST y bases de datos NoSQL. |

## 4.3. Imágenes, fotografías y vínculos

Las imágenes, fotografías y vínculos son recursos fundamentales en la construcción de contenido web, ya que complementan la información, facilitan la navegación y contribuyen a la experiencia del usuario. Su incorporación requiere considerar aspectos técnicos que permitan optimizar la carga de los recursos visuales y establecer conexiones adecuadas entre diferentes contenidos y páginas.

En el caso de las imágenes y fotografías, es necesario equilibrar la calidad visual y el rendimiento, evitando que archivos de gran tamaño incrementen innecesariamente los tiempos de carga. Para ello, pueden aplicarse diferentes técnicas de optimización:

- **Compresión adaptativa.** Uso de algoritmos de compresión para reducir el peso en kilobytes de los archivos ráster sin comprometer apreciablemente la calidad visual.
- **Carga diferida (lazy loading).** Uso del atributo loading="lazy" en etiquetas <img> para postergar la descarga de imágenes que se encuentran fuera de la ventana gráfica inicial (viewport) hasta que el usuario se desplaza hacia ellas.

- **Imágenes responsivas.** Implementación de la etiqueta <picture> o el atributo srcset para servir versiones comprimidas y dimensionadas según la resolución del dispositivo del usuario.

Estas técnicas pueden combinarse dentro de la estructura HTML para seleccionar diferentes versiones de una imagen según las características de visualización y diferir su carga cuando no se encuentra inicialmente visible. El siguiente ejemplo muestra el uso de <picture> para definir recursos según el tamaño de pantalla y de loading="lazy" para aplicar carga diferida a la imagen:

```
<!-- Ejemplo de imagen responsiva con fallbacks -->

<picture>

<source srcset="foto-desktop.webp" media="(min-width: 1024px)"
type="image/webp">

<source srcset="foto-mobile.webp" media="(max-width: 767px)"
type="image/webp">

<img src="foto-default.jpg" alt="Descripción accesible de la imagen"
loading="lazy">

</picture>
```

Además de optimizar los recursos visuales, la organización del contenido web requiere establecer vínculos que permitan al usuario desplazarse entre páginas, recursos o secciones específicas. Según su destino y función dentro de la navegación, los enlaces pueden clasificarse de la siguiente manera:

- **Enlaces internos**. Conectan recursos dentro del mismo dominio, fortaleciendo la estructura de navegación y la transferencia de autoridad SEO.
- **Enlaces externos.** Redirigen a dominios de terceros. Deben incluir atributos de seguridad como rel="noopener noreferrer" al utilizar target="_blank" para prevenir ataques de secuestro de pestaña (tabnabbing).
- **Enlaces de anclaje.** Facilitan la navegación intrapágina hacia secciones específicas mediante identificadores id.

## 4.4. Diseño visual y factores del diseño visual

El diseño visual es el lenguaje gráfico mediante el cual se organizan y presentan los elementos de una interfaz para comunicar la identidad de marca, establecer la jerarquía de la información y orientar los puntos de interacción. Su aplicación busca que los componentes visuales mantengan coherencia, legibilidad y organización, de manera que el usuario pueda reconocer con facilidad la importancia y función de cada elemento.

Para lograrlo, el diseño visual considera diferentes factores relacionados con el uso del color, la tipografía, el contraste y la distribución espacial de los elementos. Entre los principales se encuentran los siguientes:

### A. Teoría del color y paleta cromática

El uso planificado del color permite establecer jerarquías, diferenciar elementos y mantener coherencia en la interfaz. Una distribución de referencia puede organizarse de la siguiente manera:

- **Color dominante (60 %):** define el fondo y las áreas neutras de la aplicación.
- **Color secundario (30 %):** utilizado en tarjetas, bloques de información y barras laterales.
- **Color de acento (10 %):** utilizado para destacar elementos de llamada a la acción (Call to Action – CTA), como botones de compra o confirmación.

### B. Jerarquía tipográfica

Establece diferencias claras de tamaño, peso (font-weight) y contraste entre títulos (h1), subtítulos (h2, h3) y cuerpo de texto para guiar la lectura y facilitar la identificación de los diferentes niveles de información. Una jerarquía bien definida permite distinguir contenidos principales y secundarios, favorece la lectura secuencial y mantiene la consistencia visual entre las diferentes secciones de la interfaz.

### C. Contraste y accesibilidad (WCAG)

Considera relaciones de contraste entre el texto y el fondo que favorezcan la legibilidad y permitan que la información pueda ser percibida por usuarios con diferentes capacidades visuales. Como referencia, se establecen relaciones mínimas de 4.5:1 para texto normal y 3:1 para texto grande, de acuerdo con los criterios de accesibilidad aplicables. El contraste también permite diferenciar componentes de la

interfaz y facilitar la identificación visual de información relevante, evitando combinaciones de colores que dificulten su percepción o comprensión.

**D. Espaciado y alineación (grid system)**

El uso de retículas y módulos de espaciado permite organizar los componentes de manera consistente y mantener relaciones visuales uniformes entre márgenes, rellenos (padding) y alineaciones. Una distribución adecuada de los espacios facilita la separación y agrupación visual de los contenidos, evita la saturación de la interfaz y contribuye a establecer un orden claro entre sus elementos. Asimismo, favorece la adaptación de la composición a diferentes tamaños de pantalla, manteniendo la coherencia del diseño.

## 4.5. Licencias para el uso y publicación de recursos

El uso y publicación de recursos digitales, como imágenes, código, fuentes tipográficas, audios e ilustraciones, está sujeto a condiciones de propiedad intelectual y a diferentes tipos de licencias. Estas determinan las posibilidades de usar, copiar, modificar, distribuir o incorporar un recurso dentro de una aplicación web, así como las obligaciones que debe cumplir quien lo utiliza.

Por esta razón, antes de integrar recursos creados por terceros es necesario identificar la licencia aplicable y verificar sus condiciones de uso, especialmente en aspectos relacionados con la atribución, la modificación, la redistribución y el uso comercial. La siguiente tabla presenta algunas de las licencias utilizadas en recursos digitales y de código, junto con sus principales permisos, restricciones y ejemplos de aplicación:

**Tabla 8.** Matriz de licencias estándar para recursos digitales en la web

| Tipo de licencia | Permisos principales | Restricciones / obligaciones | Ejemplo de uso |
|---|---|---|---|
| CC0. | Copiar, modificar, distribuir y usar comercialmente sin costo. | Ninguna: se renuncia explícitamente a los derechos de autor. | Recursos publicados expresamente bajo CC0. |
| Creative Commons (CC BY). | Uso libre para fines comerciales y no comerciales. | Requisito de atribución: se debe citar obligatoriamente al autor original. | Fotografías, ilustraciones y otros recursos publicados bajo CC BY. |
| CC BY-NC. | Uso libre para modificaciones y distribución. | No comercial: prohibido su uso en productos que generen ingresos. | Recursos educativos o de investigación publicados bajo esta licencia. |
| CC BY-SA. | Uso y modificación libres. | Compartir igual: las obras derivadas deben publicarse bajo la misma licencia. | Contenidos y recursos abiertos publicados bajo CC BY-SA (Wikipedia). |
| Licencia MIT (código). | Uso, copia, modificación, fusión y venta del código. | Incluir: aviso de *copyright* y la exención de responsabilidad original. | Librerías y proyectos de código abierto |

| Tipo de licencia | Permisos principales | Restricciones / obligaciones | Ejemplo de uso |
|---|---|---|---|
| | | | distribuidos bajo licencia MIT (React). |
| Licencia GNU GPL v3. | Uso y modificación libre del código fuente. | Copyleft: si se distribuye una versión derivada, debe mantenerse la licencia GPL v3 y facilitarse el código fuente correspondiente. | Software y proyectos distribuidos bajo GNU GPL v3 (WordPress). |

## 4.6. Configuración y administración de bases de datos de contenido web

El contenido web dinámico requiere mecanismos que permitan almacenar, consultar, actualizar y recuperar información de manera eficiente. Para ello, las aplicaciones pueden utilizar motores de bases de datos relacionales o no relacionales, cuya selección depende de aspectos como la estructura de los datos, los requerimientos de consistencia y las necesidades de la aplicación.

En una arquitectura web, la base de datos se integra generalmente con los servicios del backend, encargados de procesar las solicitudes y gestionar el acceso a la información. El siguiente esquema representa de manera simplificada esta interacción entre el cliente, el servicio backend y la base de datos:

[ Cliente / web ] ◄— HTTP / JSON —► [ Servicio backend / ORM ] ◄— SQL / NoSQL —► [ Base de datos ]

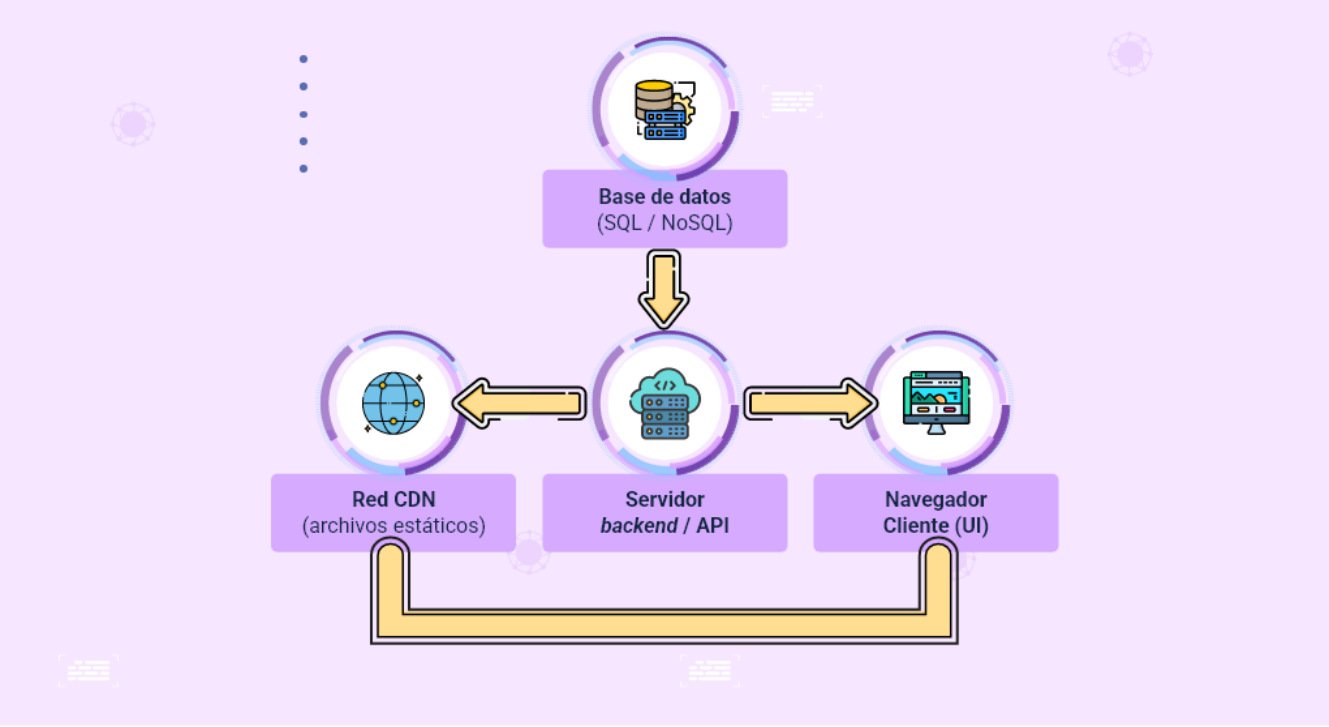
Para gestionar el contenido almacenado, pueden emplearse diferentes tipos de bases de datos y herramientas de acceso a la información, de acuerdo con las características y necesidades de cada aplicación:

- **Bases de datos relacionales (SQL)**
  - Motores comunes: PostgreSQL, MySQL y MariaDB.
  - Estructura: esquemas organizados en tablas, filas y columnas relacionadas mediante claves primarias (PK) y foráneas (FK).
  - Caso de uso: sistemas que requieren alta consistencia transaccional y cumplimiento de propiedades ACID, como plataformas de comercio electrónico o aplicaciones financieras.
- **Bases de datos no relacionales (NoSQL):**
  - Motores comunes: MongoDB, CouchDB y Firebase Firestore.
  - Estructura: modelos flexibles que pueden organizar la información mediante documentos, pares clave-valor u otras estructuras, según el tipo de base de datos utilizado.
  - Caso de uso: gestores de contenido (CMS), blogs, redes sociales y aplicaciones que manejan grandes volúmenes de información con estructuras variables.
- **Mapeo objeto-relacional (ORM) y mapeo objeto-documento (ODM):**
  - Herramientas comunes: Prisma, Sequelize, Mongoose y Entity Framework.
  - Estructura: utilizan objetos y modelos definidos en el código de la aplicación para representar y gestionar los datos almacenados, facilitando la interacción con la base de datos.

- Caso de uso: aplicaciones que requieren gestionar operaciones de consulta, inserción, actualización y eliminación de datos desde el código, reduciendo la necesidad de trabajar directamente con instrucciones específicas del motor de base de datos.

La selección de estas tecnologías y su integración con los demás componentes de una aplicación forman parte de una arquitectura más amplia de gestión de contenido. La siguiente figura sintetiza esta relación al mostrar la interacción entre la persistencia de datos, el procesamiento realizado en el backend, la distribución de recursos estáticos y la presentación del contenido en el navegador:

**Figura 3.** Ecosistema de gestión, procesamiento y distribución de contenido web estático y dinámico



# 5. Diseño y desarrollo web

El diseño y desarrollo web comprende el conjunto de disciplinas, protocolos de comunicación, arquitecturas de software, estándares visuales y técnicas de maquetación requeridas para la construcción de plataformas, portales y aplicaciones web modernas. Esta área integra la lógica del lado del servidor (backend), la representación e interactividad del lado del cliente (frontend) y la infraestructura de transmisión segura de datos a través de la red global de internet.

## 5.1. Procedimientos y protocolos de diseño y desarrollo web

La web opera principalmente sobre un modelo cliente-servidor, en el que diferentes protocolos y estándares permiten establecer la comunicación, transmitir información y acceder a recursos a través de la red. En este modelo, el cliente realiza solicitudes al servidor y este procesa las peticiones y devuelve las respuestas correspondientes. El siguiente esquema representa de manera simplificada este intercambio:

[ Cliente / browser ] — Petición HTTP / HTTPS (GET, POST, PUT, DELETE) —► [ Servidor web ]

◄— Respuesta HTTP (Status 200, 404, 500 + Payload) ——

- **Protocolos de comunicación fundamentales**

La comunicación entre clientes, servidores y otros componentes de la red se apoya en diferentes protocolos y sistemas, cada uno con una función específica. Entre los principales se encuentran los siguientes:

- **HTTP (Hypertext Transfer Protocol).** Protocolo de la capa de aplicación utilizado para la transferencia de recursos y datos en la web. Funciona mediante un esquema de petición y respuesta (request / response).
- **HTTPS (Hypertext Transfer Protocol Secure).** Versión segura de HTTP que utiliza TLS (Transport Layer Security) para proteger la comunicación y contribuir a garantizar la confidencialidad, integridad y autenticidad de los datos transmitidos.
- **TCP/IP (Transmission Control Protocol/Internet Protocol).** Conjunto de protocolos fundamentales para la comunicación en redes, que intervienen en la transmisión y el direccionamiento de la información entre los dispositivos conectados.
- **DNS (Domain Name System).** Sistema jerárquico de resolución de nombres que permite asociar nombres de dominio legibles por las personas, como www.sena.edu.co, con las direcciones IP correspondientes.

- **Métodos y códigos de estado HTTP**

Para la construcción de servicios web e interfaces de programación de aplicaciones (API), se utilizan métodos HTTP estandarizados para indicar las operaciones que se realizarán sobre los recursos. Las respuestas del servidor incluyen códigos de estado que informan el resultado de la solicitud. La siguiente tabla presenta algunos de los métodos y códigos más utilizados:

**Tabla 9.** Métodos HTTP y códigos de estado estándar en el desarrollo web

| Método / categoría | Acción principal | Código de estado común | Significado |
|---|---|---|---|
| GET. | Consultar o recuperar un recurso sin modificarlo. | 200 OK. | Solicitud procesada exitosamente. |
| POST. | Enviar datos para crear un nuevo recurso en el servidor. | 201 Created. | Recurso creado con éxito. |
| PUT. | Actualizar completamente un recurso existente. | 200 OK / 204 No Content. | Actualización realizada correctamente. |
| PATCH. | Actualizar parcialmente un recurso existente. | 200 OK. | Modificación parcial procesada. |
| DELETE. | Eliminar un recurso del servidor. | 200 OK / 204 No Content. | Eliminación procesada correctamente. |
| Error del cliente. | Indica un problema relacionado con la solicitud enviada. | 400 Bad Request / 401 Unauthorized / 404 Not Found. | Solicitud incorrecta, ausencia de autenticación válida o recurso no encontrado. |

| Método / categoría | Acción principal | Código de estado común | Significado |
|---|---|---|---|
| Error del servidor. | Indica un problema ocurrido al procesar la solicitud en el servidor. | 500 Internal Server Error / 503 Service Unavailable. | Error interno o servicio temporalmente no disponible. |

## 5.2. Técnicas de diseño y estructura visual

La construcción visual de una aplicación web requiere una estructura que permita organizar el contenido, definir su presentación y establecer su comportamiento e interactividad. En este proceso, HTML5 proporciona la estructura y el contenido, CSS permite controlar la presentación visual y JavaScript incorpora comportamientos dinámicos e interacción con el usuario. A partir de esta integración pueden aplicarse diferentes técnicas para organizar los elementos de la interfaz y adaptar su presentación a distintos dispositivos.

Entre las técnicas utilizadas para estructurar y distribuir visualmente los elementos de una interfaz web se encuentran las siguientes:

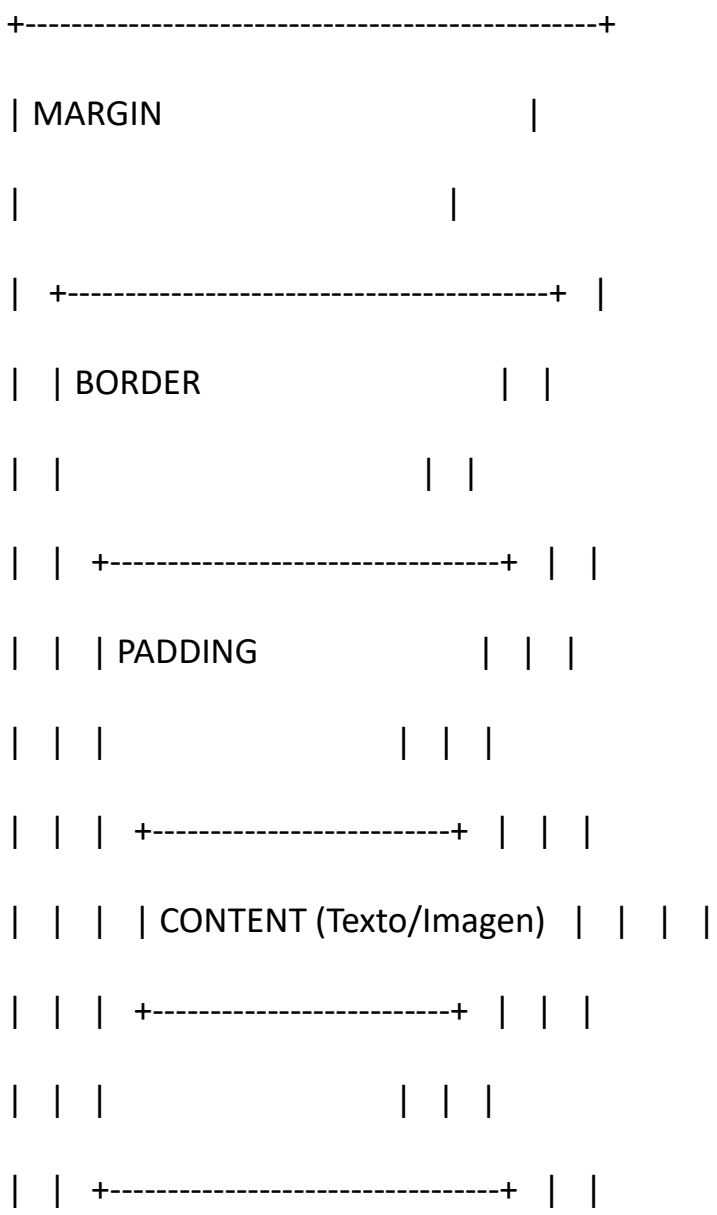
### A. Modelo de caja CSS (CSS box model)

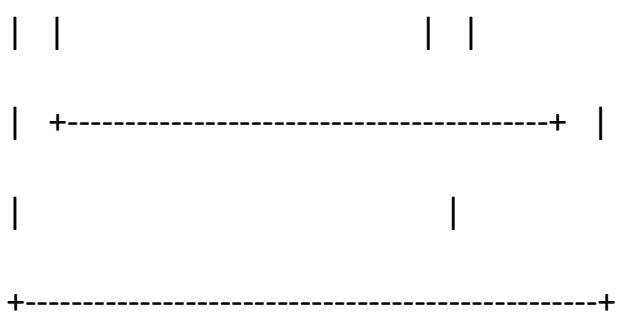
Cada elemento HTML se representa visualmente como una caja rectangular compuesta por áreas que determinan el espacio ocupado por el contenido y su relación con los demás elementos:

- **Contenido (content):** área donde se presenta el texto, las imágenes u otros elementos.

- **Relleno (padding):** espacio entre el contenido y el borde.
- **Borde (border):** línea que rodea el contenido y el relleno.
- **Margen (margin):** espacio exterior que separa la caja de los elementos vecinos.

El siguiente esquema representa la disposición de estas áreas, desde el margen exterior hasta el contenido ubicado en el centro del elemento:





**B. Diseño web adaptable**

Técnica de diseño que permite adaptar la distribución y presentación de una interfaz a diferentes tamaños y características de pantalla. Su implementación combina recursos que permiten dimensionar los elementos de manera flexible y modificar su disposición según el dispositivo. Entre sus principales componentes se encuentran:

- **Cuadrículas fluidas:** empleo de unidades relativas, como %, vw, rem y ch, que permiten establecer dimensiones flexibles en lugar de depender exclusivamente de valores fijos en píxeles (px).
- **Imágenes flexibles:** aplicación de reglas CSS, como `max-width: 100 %`, para que las imágenes puedan ajustarse al espacio disponible y evitar desbordamientos dentro de su contenedor.
- **Consultas de medios:** permiten aplicar reglas de estilo de acuerdo con determinadas características del dispositivo o de la ventana de visualización. Pueden utilizar puntos de interrupción (`breakpoints`) para modificar la distribución de los elementos según el ancho, la orientación u otras condiciones definidas.

## 5.3. Protocolos de seguridad de la información

La seguridad constituye un principio fundamental en el desarrollo de aplicaciones web y debe considerarse desde las etapas iniciales del diseño y la implementación (security by design). Las aplicaciones pueden estar expuestas a diferentes vulnerabilidades y ataques orientados al acceso no autorizado, la manipulación de información, el compromiso de sesiones o la afectación de la disponibilidad de los servicios. Por esta razón, es necesario incorporar medidas preventivas y mecanismos de protección durante todo el proceso de desarrollo.

- **Vulnerabilidades comunes en aplicaciones web**

Entre las vulnerabilidades que pueden afectar a una aplicación se encuentran:

- **Inyección SQL (SQLi)**

Ocurre cuando datos de entrada no controlados se incorporan de manera insegura en consultas SQL, permitiendo que un atacante altere su comportamiento y pueda acceder o manipular información de la base de datos. Para reducir este riesgo se emplean mecanismos como las consultas parametrizadas o preparadas (prepared statements) y prácticas seguras de acceso a datos mediante herramientas ORM.

- **Cross-Site Scripting (XSS)**

Consiste en la introducción de contenido o scripts maliciosos que posteriormente pueden ejecutarse en el navegador de otros usuarios. Este tipo de ataque puede comprometer información o acciones asociadas a la sesión. Su prevención requiere aplicar medidas como la codificación adecuada de las salidas según el contexto, el tratamiento seguro de los datos ingresados y políticas como Content Security Policy (CSP).

- **Falsificación de solicitudes entre sitios (Cross-Site Request Forgery – CSRF)**

Busca aprovechar una sesión autenticada para provocar que el navegador del usuario envíe una solicitud no deseada a una aplicación. Entre las medidas de protección pueden utilizarse tokens anti-CSRF, una configuración apropiada del atributo SameSite de las cookies y otras validaciones asociadas a las solicitudes.

Además de las medidas específicas para prevenir estas vulnerabilidades, las aplicaciones web requieren mecanismos de seguridad que protejan las comunicaciones, la autenticación, las sesiones y la interacción entre diferentes orígenes. La siguiente tabla presenta algunos de estos mecanismos, su función principal y ejemplos de implementación técnica:

**Tabla 10.** Mecanismos e infraestructura de seguridad en el desarrollo web

| Mecanismo de seguridad | Función principal | Implementación técnica |
|---|---|---|
| Certificados TLS. | Proteger la comunicación entre el cliente y el servidor mediante cifrado y autenticación del servidor. | Uso de HTTPS mediante certificados digitales emitidos por autoridades de certificación (CA) o mecanismos de confianza equivalentes. |
| Autenticación basada en tokens. | Permitir la autenticación y autorización mediante credenciales o tokens utilizados por la aplicación. | Uso de mecanismos como JSON Web Tokens (JWT) y estándares o protocolos como OAuth 2.0 / |

| Mecanismo de seguridad | Función principal | Implementación técnica |
|---|---|---|
| | | OpenID Connect, según los requerimientos de la aplicación. |
| Encabezados de seguridad HTTP. | Establecer políticas de seguridad que el navegador debe aplicar al procesar los recursos de la aplicación. | Encabezados como Strict-Transport-Security (HSTS), X-Frame-Options y X-Content-Type-Options. |
| CORS (Cross-Origin Resource Sharing). | Controlar qué orígenes pueden acceder a determinados recursos del servidor desde el navegador. | Configuración de encabezados CORS, como Access-Control-Allow-Origin, de acuerdo con los orígenes autorizados. |

## 5.4. Técnicas de maquetación

La maquetación web permite organizar y distribuir los elementos de una interfaz para establecer su posición, alineación y relación dentro de la página. CSS proporciona diferentes mecanismos para construir estas estructuras, entre los que destacan Flexbox y CSS Grid, utilizados para controlar la disposición de los componentes de acuerdo con las necesidades del diseño.

Aunque ambos permiten organizar elementos, se diferencian principalmente por la forma en que gestionan su distribución. El siguiente esquema representa de manera

simplificada esta diferencia: Flexbox trabaja principalmente sobre un eje, mientras que CSS Grid permite organizar elementos mediante filas y columnas:

FLAT LAYOUT (Flexbox - 1D)

+---+---+---+

| 1 | 2 | 3 |  (Fila única)

+---+---+---+

GRID LAYOUT (CSS Grid - 2D)

+-----+-----+-----+

| 1.1 | 1.2 | 1.3 |

+-----+-----+-----+

| 2.1 | 2.2 | 2.3 |

+-----+-----+-----+

A partir de esta diferencia, cada técnica ofrece características y propiedades específicas para controlar la disposición de los elementos:

- **Flexible box layout**
    - Orientación: unidimensional, ya que permite organizar los elementos principalmente a lo largo de un eje, como una fila o una columna.
    - Fortalezas: facilita la alineación, la distribución dinámica del espacio y la organización de componentes como barras de navegación, listas de tarjetas y grupos de botones.

- Las propiedades clave incluyen display: flex, justify-content, align-items, flex-direction y flex-wrap.

- **CSS grid layout**
  - Orientación: bidimensional, ya que permite organizar los elementos mediante filas y columnas.
  - Fortalezas: facilita la construcción de estructuras de página y distribuciones que requieren controlar simultáneamente ambas dimensiones, como encabezados, barras laterales, áreas de contenido y pies de página.
  - Las propiedades clave incluyen display: grid, grid-template-columns, grid-template-rows, gap y grid-area.

## 5.5. Parámetros de visualización

Los parámetros de visualización permiten evaluar aspectos relacionados con la presentación, el rendimiento y la experiencia del usuario durante la carga e interacción con una aplicación web. Su análisis facilita la identificación de elementos que pueden afectar la velocidad de visualización, la capacidad de respuesta y la estabilidad del contenido en el navegador.

- **Indicadores clave de rendimiento** web **(Core Web Vitals de Google)**

Los Core Web Vitals permiten evaluar aspectos de la experiencia del usuario mediante métricas relacionadas con la carga, la capacidad de respuesta y la estabilidad visual. Entre las principales se encuentran:

- **Largest Contentful Paint (LCP).** Mide el tiempo que tarda en renderizarse el elemento de contenido más grande visible en la ventana gráfica, como una imagen o un bloque de texto principal. Un valor adecuado corresponde a 2,5 segundos o menos.
- **Interaction to Next Paint (INP).** Mide la capacidad de respuesta de una página ante las interacciones del usuario, como clics, toques de pantalla o entradas mediante el teclado. Un valor adecuado corresponde a 200 milisegundos o menos.
- **Cumulative Layout Shift (CLS).** Mide la estabilidad visual de la página a partir de los cambios inesperados en la posición de los elementos durante su uso. Un valor adecuado corresponde a 0,1 o menos.

- **Optimización del renderizado en el navegador**

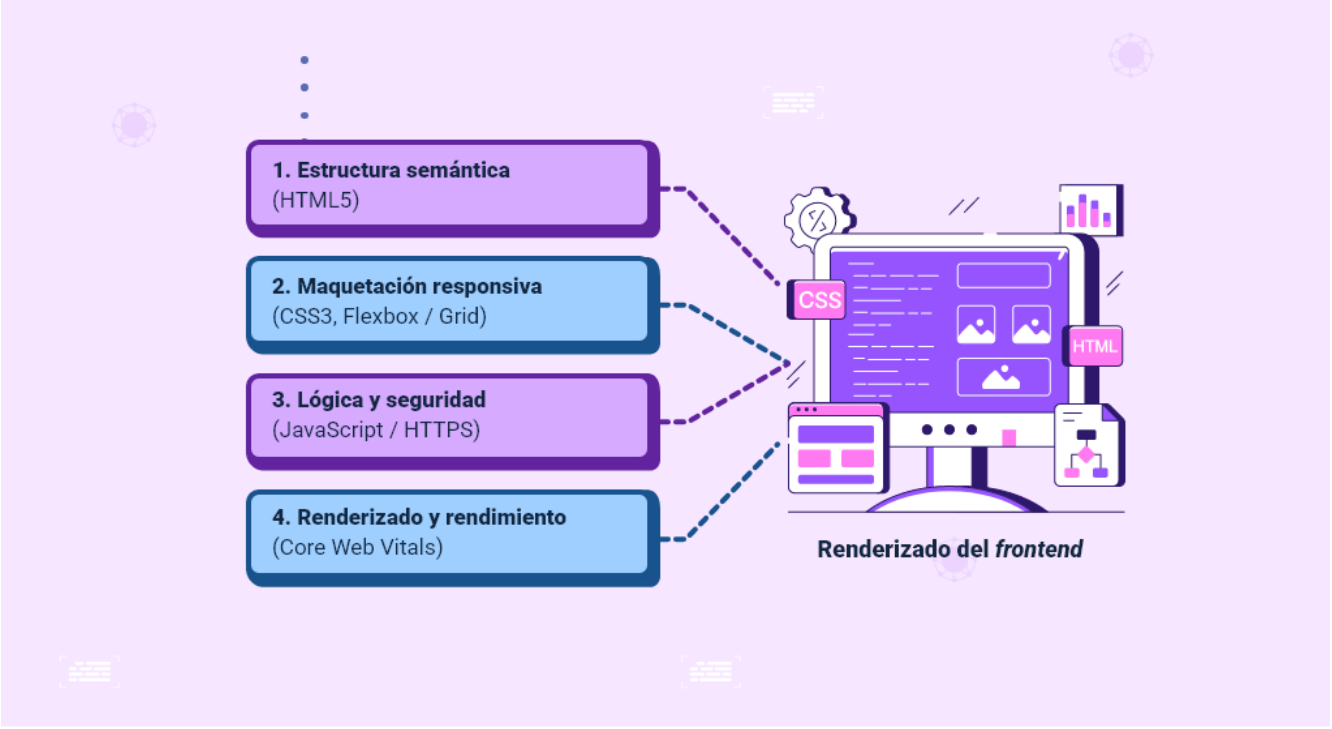
Para favorecer el rendimiento de una aplicación web, pueden aplicarse técnicas orientadas a reducir el tamaño de los recursos y facilitar la presentación inicial del contenido. Entre ellas se encuentran:

- **Minificación y agrupación (bundling).** La minificación reduce el tamaño de archivos .js y .css mediante la eliminación o transformación de elementos innecesarios para su ejecución, mientras que la agrupación permite organizar los recursos generados durante el proceso de construcción de la aplicación. Estas tareas pueden gestionarse mediante herramientas como Vite, Webpack o Turbopack.
- **CSS crítico (critical CSS).** Consiste en priorizar los estilos necesarios para presentar el contenido inicialmente visible, reduciendo la

dependencia de la descarga y procesamiento de hojas de estilo completas antes de mostrar esa parte de la página.

Los aspectos relacionados con el rendimiento y el renderizado se integran con otros componentes técnicos que intervienen en la construcción de una aplicación web. La siguiente figura sintetiza esta relación mediante una arquitectura por capas que articula la estructura semántica, la maquetación responsiva, la lógica y seguridad, y el renderizado y rendimiento de la interfaz:

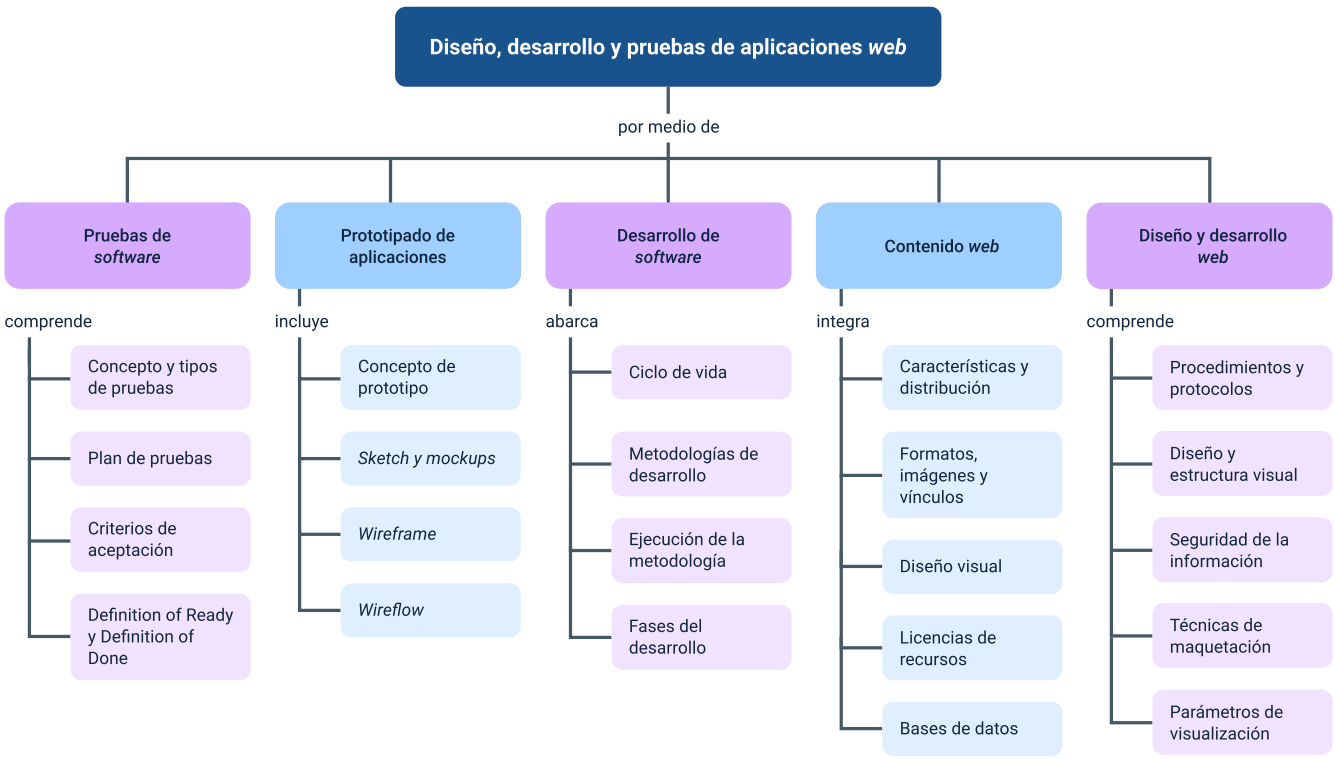
**Figura 4.** Arquitectura integrada del proceso de diseño, maquetación, seguridad y renderizado web



## Síntesis

La construcción de aplicaciones web integra conocimientos de desarrollo de software, prototipado, gestión de contenidos, seguridad y aseguramiento de la calidad. Este componente formativo aborda el ciclo de vida del software y las metodologías de desarrollo, así como la elaboración de sketches, wireframes, mockups y wireflows.

Además, comprende la gestión de contenidos multimedia y licencias, la configuración de bases de datos, los protocolos de comunicación y seguridad, así como las técnicas de diseño y maquetación web. Finalmente, incorpora el diseño de planes de pruebas, los criterios de aceptación y conceptos como Definition of Ready y Definition of Done, orientados a la construcción de soluciones confiables y acordes con los requerimientos establecidos.



## Glosario

**API (Application Programming Interface):** conjunto de reglas y mecanismos que permite la comunicación e intercambio de información entre diferentes aplicaciones, sistemas o componentes de software.

**Backlog:** lista organizada y priorizada de funcionalidades, requerimientos, mejoras o tareas pendientes asociadas con el desarrollo de un producto de software.

**Caso de prueba (test case):** conjunto de condiciones, datos de entrada, pasos de ejecución y resultados esperados definidos para verificar el comportamiento de una funcionalidad específica del software.

**Gherkin:** lenguaje estructurado utilizado para describir escenarios de comportamiento mediante expresiones como Dado, Cuando y Entonces, facilitando la formulación de criterios de aceptación verificables.

**OWASP (Open Worldwide Application Security Project):** organización internacional sin fines de lucro orientada a mejorar la seguridad del software mediante proyectos, herramientas, metodologías y recursos abiertos relacionados con la seguridad de las aplicaciones.

**Pruebas unitarias (unit testing):** nivel de prueba en el que se verifican de manera aislada unidades o componentes individuales del software, como funciones, métodos o módulos.

**QA (Quality Assurance):** conjunto de actividades orientadas a asegurar la calidad del software mediante la prevención de defectos y la aplicación sistemática de procesos, estándares y prácticas de calidad.

**Scope creep:** incremento progresivo y no controlado del alcance inicialmente establecido para un proyecto o producto, debido a la incorporación de requisitos o funcionalidades adicionales.

**Staging:** entorno previo a producción utilizado para desplegar y validar una aplicación en condiciones similares a las del entorno real antes de su publicación definitiva.

**Tabnabbing:** técnica de ataque en la que una pestaña o página abierta puede ser utilizada para redirigir al usuario o modificar el contenido mostrado con fines maliciosos.

## Referencias bibliográficas


Agile Alliance. (2023). Agile Glossary: Definition of Done and Definition of Ready. Agile Alliance Resource Library.

Creative Commons Organization. (2023). About CC Licenses: About the Licenses. Creative Commons Legal Code.

International Software Testing Qualifications Board (ISTQB). (2023). Syllabus Probador Certificado Nivel Básico (CTFL) v4.0. ISTQB Official Documents.

MDN Web Docs - Mozilla Developer Network. (2024). Aprender desarrollo web: Estructuración y diseño frontend. Mozilla Foundation.

OWASP Foundation. (2021). OWASP Top 10:2021 - Los diez riesgos de seguridad de aplicaciones web más críticos. Open Web Application Security Projectu.

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum: La Guía Definitiva de Scrum: Las Reglas del Juego. Scrum.org.

World Wide Web Consortium (W3C). (2023). Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation.

# Créditos

| Nombre | Cargo | Centro de Formación y Regional |
|---|---|---|
| Claudia Johanna Gómez Pérez | Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales | Centro Agroturístico - Regional Santander |
| Diana Rocío Possos Beltrán | Responsable de línea de producción | Centro de Comercio y Servicios - Regional Tolima |
| Sebastian Trujillo Afanador | Experto temático | Centro de Comercio y Servicios - Regional Tolima |
| Andrés Felipe Velandia Espitia | Evaluador instruccional | Centro de Comercio y Servicios - Regional Tolima |
| Juan Daniel Polanco Muñoz | Diseñador de contenidos digitales | Centro de Comercio y Servicios - Regional Tolima |
| Francisco José Vásquez Suárez | Desarrollador `full stack` | Centro de Comercio y Servicios - Regional Tolima |
| Ernesto Navarro Jaimes | Animador y productor audiovisual | Centro de Comercio y Servicios - Regional Tolima |
| María Fernanda Pineda Mora | Evaluadora de contenidos inclusivos y accesibles | Centro de Comercio y Servicios - Regional Tolima |
| Jorge Bustos Gómez | Validador y vinculador de recursos educativos digitales | Centro de Comercio y Servicios - Regional Tolima |